

Concurrency, Fault Tolerance & Caching – Short Answers

Q1. Thread vs Async

Threads are suitable for CPU-bound tasks like data processing. Async/non-blocking I/O is better for I/O-bound tasks like network or database calls.

Q2. Virtual Threads (Java 21)

Virtual Threads are lightweight threads managed by JVM. They are ideal for high-concurrency I/O tasks but not CPU-intensive workloads.

Q3. Lock vs CAS

Locks block threads and are simpler. CAS is non-blocking and performs better under high contention but is more complex.

Q4. Deadlock Conditions

Deadlock requires mutual exclusion, hold-and-wait, no preemption, and circular wait. Lock ordering breaks circular wait.

Q5. Thread Pool Sizing

Formula: Threads = CPU cores $\times$ (1 + wait time / compute time). CPU-bound $\approx$ number of cores; I/O-bound needs more threads.

Q6. Exponential Backoff with Jitter

It increases retry delay exponentially. Jitter adds randomness to avoid synchronized retries.

Q7. Circuit Breaker

CLOSED allows traffic, OPEN blocks requests, HALF-OPEN tests recovery. Transitions depend on failure and success rates.

Q8. Exactly-Once Semantics

Exactly-once delivery is impossible due to network failures. Exactly-once processing is achieved using idempotency and transactions.

Q9. Caching Patterns

Cache-Aside is flexible, Write-Through ensures consistency, Write-Behind improves performance but risks data loss.

Q10. Cache Stampede

Cache stampede happens when many requests hit DB after cache expiration. Solutions include locking and early refresh.